Universitat de Girona

CONFIGURACIÓ I MANTENIMENT DE XARXES I SERVEIS D'INTERNET

PROJECTE 1

Funcionament dels protocols i arquitectura de serveis *Streaming*

HLS i Adaptive Bit Rate

MEMÒRIA

Darius Natan Santa, u1994947, natan.d.santa@gmail.com

Girona, novembre de 2026

Índex

1 Introducció i objectius **3**

2 Conceptes bàsics **3**

2.1 Entorn tècnic (part comuna) . . . 3

2.2 Servidor recursiu amb BIND9 (part comuna) . . . 3

2.3 Tècniques de *streaming* anteriors a HLS (part específica) . . . 4

2.4 HLS (*HTTP Live Streaming*) . . . 4

2.4.1 Què és i per què s'utilitza . . . 4

2.4.2 Procés general . . . 4

2.4.3 Els tres components . . . 5

2.4.4 Com interactuen els components . . . 6

2.5 FFmpeg i HLS . . . 6

2.6 *Adaptive Bit Rate* (ABR) i EWMA . . . 7

2.6.1 Per què una mitjana mòbil exponencial . . . 8

2.6.2 EWMA ràpida i EWMA lenta . . . 8

3 Escenari 1: descripció **8**

3.1 Part comuna: infraestructura de xarxa de l'organització . . . 8

3.2 Part específica: plataforma *MeTube* . . . 9

4 Escenari 1: construcció **10**

4.1 Part comuna: màquines virtuals, *router* i DNS . . . 10

4.1.1 Màquines i *router* . . . 10

4.1.2 Problema de connectivitat entre màquines virtuals . . . 11

4.1.3 Servidor DNS recursiu (BIND9) . . . 11

4.2 Part específica: construcció de MeTube . . . 11

4.2.1 Cronologia del treball . . . 11

4.2.2 Instal·lació i execució . . . 12

4.2.3 Limitació de la xarxa amb `tc` . . . 12

4.2.4 Generació de les gràfiques . . . 12

4.2.5 Execució de l'experiment . . . 12

5 Escenari 1: anàlisi **13**

5.1 Part comuna: proves de funcionament . . . 13

5.2 Part específica: comprovació del funcionament d'HLS . . . 13

5.3 Part específica: experiment de l'ABR . . . 14

5.3.1 Disseny de l'experiment 14
5.3.2 Dades registrades 14
5.3.3 Resultats 14

6 Resum, conclusions i treball futur 16
6.1 Què s'ha fet 16
6.2 Problemes trobats 16
6.3 Conclusions i lliçons apreses 16
6.4 Treball futur 16

7 Treball en grup, dedicació i suggeriments 17

8 Ús d'eines d'Intel·ligència Artificial Generativa (IAG) 17

Bibliografia 17

A Annex A: perfil de limitació de xarxa utilitzat 18

1 Introducció i objectius

L'objectiu d'aquest treball és analitzar els protocols utilitzats pels serveis de *streaming* de vídeo, observar les tècniques i els mètodes que fan servir i comprovar-ne el funcionament amb una implementació pròpia. El treball se centra en el protocol *HTTP Live Streaming* (HLS) i en l'adaptació de la qualitat a la connexió, coneguda com a *Adaptive Bit Rate* (ABR).

Els objectius concrets són:

- Aprofundir en el funcionament dels serveis de *streaming*.
- Entendre per què HLS va suposar un canvi important respecte a les tècniques anteriors.
- Crear una plataforma de *streaming* pròpia (anomenada *MeTube*) per comprovar com funciona el protocol.
- Estudiar què és l'ABR i com l'implementa HLS, mesurant-ne el comportament amb condicions de xarxa que canvien al llarg del temps.

Organització de la memòria. El Projecte 1 té dues parts. La **part comuna** és la infraestructura de xarxa d'una organització que, junt amb les dels companys de classe, forma una "petita Internet" dins del laboratori. La **part específica** és el treball propi del tema triat, que en aquest cas és el *streaming* amb HLS. Al llarg de la memòria les dues parts es distingeixen de forma explícita: les seccions 3, 4 i 5 tenen una subsecció per a cada una ("Part comuna" i "Part específica").

En aquesta memòria es distingeix el que s'explica a partir de documentació externa (amb la citació corresponent) del que s'ha observat durant el projecte, que es presenta com a resultat propi.

2 Conceptes bàsics

2.1 Entorn tècnic (part comuna)

L'escenari es construeix amb màquines virtuals de *VirtualBox* [1] i un *router*. Les dues màquines virtuals, client i servidor, utilitzen **Arch Linux** [2], una distribució de Linux que he escollit perquè la conec bé. No es dedica més espai al sistema operatiu: el que importa és com s'ha fet servir, i això es descriu a les seccions 3 i 4.

El *router* utilitza **OpenWrt** [3], un *firmware* de codi obert basat en Linux per a *routers*. Té una interfície web, anomenada LuCI, per configurar-lo. Vaig triar OpenWrt per experimentar amb aquest *firmware*, que m'interessava investigar.

2.2 Servidor recursiu amb BIND9 (part comuna)

El servidor de noms de la meva organització és un servidor recursiu configurat amb **BIND9** [4]. Aquest servidor pregunta al servidor autoritatiu que va fer en Pau, un company de classe, on es gestiona el nom públic del meu servidor web (`www.ponent.cat`).

2.3 Tècniques de *streaming* anteriors a HLS (part específica)

Per entendre per què HLS és útil, convé veure els enfocaments anteriors i els seus problemes. El *streaming* de vídeo i l'adaptació de la qualitat també s'expliquen a [5].

- ***HTTP Progressive Download***: el vídeo es descarrega com un fitxer normal. Només hi ha una qualitat i la reproducció es pot interrompre si la connexió no va prou ràpid.
- **RTSP**/**RTP** (*Real Time Streaming*): funciona sobre UDP, de manera que la pèrdua de paquets afecta la qualitat. Els tallafocs i les xarxes corporatives sovint bloquejaven aquest trànsit, i requeria una infraestructura especialitzada i complexa, amb poca escalabilitat.
- **RTMP** (associat a Adobe Flash Player): depenia d'una tecnologia propietària, normalment amb infraestructura especialitzada, i tenia problemes de compatibilitat amb dispositius mòbils (iOS no admetia Flash Player al navegador).

2.4 HLS (*HTTP Live Streaming*)

2.4.1 Què és i per què s'utilitza

HLS és un protocol de *streaming* adaptatiu basat en HTTP, desenvolupat per Apple i especificat a l'RFC 8216 [6]. La idea central és que el client **no descarrega un únic vídeo gegant**: el servidor publica una jerarquia de fitxers que permet al reproductor anar demanant només el que necessita.

Els avantatges d'HLS respecte a RTSP o RTMP, segons s'ha estudiat durant el treball, són:

- Funciona amb un servidor HTTP corrent: no cal infraestructura nova i es poden reutilitzar servidors web i xarxes de distribució de continguts (CDN) existents.
- Va encapsulat sobre HTTP, de manera que els tallafocs i les xarxes ja el permeten i no calen ports addicionals.
- Permet adaptar la qualitat segons l'estat de la xarxa (ABR), reduint les interrupcions.
- Escala bé, perquè tot són peticions HTTP i el contingut és compatible amb mecanismes de memòria cau (*cache*), CDN i *reverse proxies*.

2.4.2 Procés general

El protocol es pot resumir en tres passos (veieu la Fig. 1):

1. **Codificació i segmentació.** El vídeo i l'àudio originals es codifiquen a diferents resolucions i *bitrates* (el *bitrate* és el nombre de bits per segon, una manera de quantificar la qualitat). Cada versió es divideix en segments curts.
2. **Generació de les llistes.** El servidor genera fitxers de text anomenats *playlists* (o *manifests*), que tenen metadades i la llista ordenada dels segments. Aquests dos primers passos només es fan una vegada, quan es puja el vídeo al servidor.
3. **Reproducció.** El client descarrega primer la llista de reproducció, la llegeix, i després va demanant els segments un per un mitjançant HTTP estàndard.

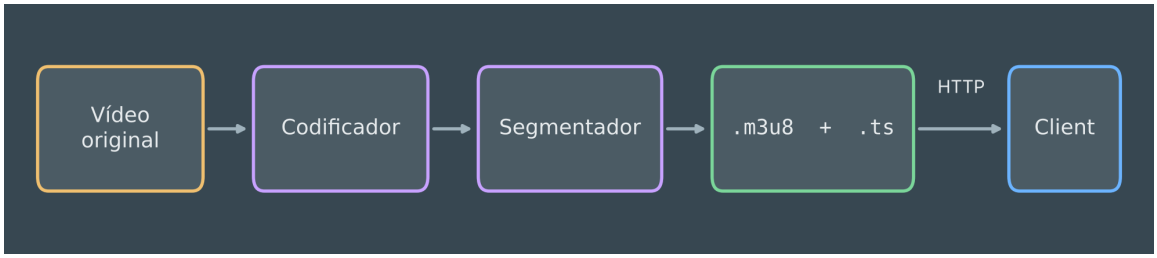


Figura 1: Flux general d'HLS: el vídeo es codifica i se segmenta, i el client en rep les llistes i els segments per HTTP.

2.4.3 Els tres components

Els tres components formen una jerarquia (veieu la Fig. 2).

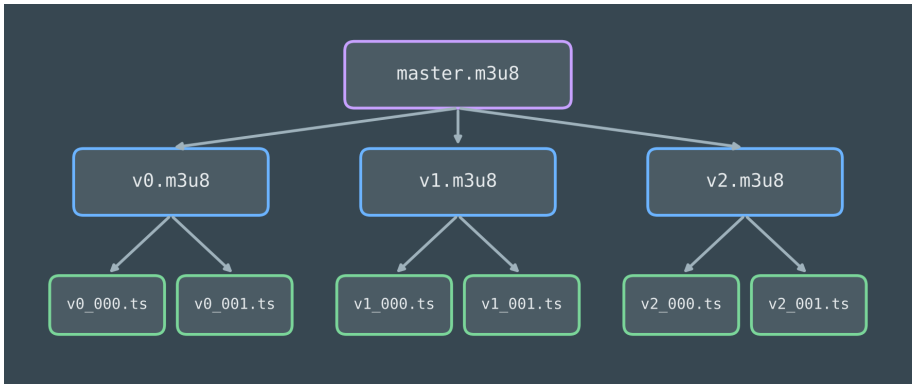


Figura 2: Jerarquia de fitxers d'HLS: *master playlist*, *media playlists* i segments.

1. *Master playlist*. És el punt d'entrada del reproductor. No conté vídeo: conté la informació de totes les variants disponibles (resolució i *bitrate*) perquè el reproductor pugui triar la que s'adapti millor a la connexió. En el nostre projecte és el fitxer `master.m3u8`. El Llistat 1 en mostra un exemple, amb una línia `#EXT-X-STREAM-INF` per a cada variant: l'atribut `BANDWIDTH` indica el *bitrate* necessari i `RESOLUTION` la resolució; la línia següent és el nom de la *playlist* d'aquella variant.

```
#EXTM3U
#EXT-X-VERSION:3

#EXT-X-STREAM-INF:BANDWIDTH=532800,RESOLUTION=426x240
v0.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1128000,RESOLUTION=854x480
v1.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2928000,RESOLUTION=1280x720
v2.m3u8
```

Llistat 1: Exemple de *master playlist* (`master.m3u8`).

2. *Media playlist*. Hi ha una per a cada qualitat (`v0.m3u8`, `v1.m3u8`...). Indica l'ordre i la durada dels segments d'aquella variant. És un fitxer de text petit; les dades pesades són els segments. Com que el vídeo és *on demand* (VOD), la *playlist* acaba amb l'etiqueta `#EXT-X-ENDLIST` (Llistat 2).

```
#EXTM3U
```

```
#EXT-X-TARGETDURATION:6
#EXT-X-PLAYLIST-TYPE:VOD

#EXTINF:6.000,
v0_000.ts
#EXTINF:6.000,
v0_001.ts
#EXTINF:6.000,
v0_002.ts
...
#EXT-X-ENDLIST
```

Llistat 2: Exemple de *media playlist* d'una qualitat (`v0.m3u8`).

3. Segments. Són les dades que realment es reprodueixen. Cada segment és un tros curt de vídeo i àudio en format MPEG-TS (extensió `.ts`). Segons la durada objectiu configurada en el projecte, haurien de ser de 6 segons. El reproductor els va demanant un darrere l'altre, la qual cosa permet començar a reproduir ràpidament (no cal esperar que es descarregui tot el fitxer) i canviar de qualitat entre segments.

2.4.4 Com interactuen els components

L'ordre de les peticions d'un client que reprodueix un vídeo és el següent:

1. El client demana la *master playlist* (`/hls/<id>/master.m3u8`).
2. Tria una variant, demana la seva *media playlist* i hi llegeix la llista de segments.
3. Demana el primer segment, el descarrega i el reprodueix; mentre es reprodueix, demana els següents.
4. Després de cada segment, pot decidir continuar amb la mateixa variant o canviar a una altra, demanant els segments següents de la *playlist* corresponent.

El servidor, en aquest procés, no calcula res: només serveix fitxers estàtics. Aquesta és una de les idees clau que s'ha descobert durant el treball, i es tracta a la Secció 2.6.

2.5 FFmpeg i HLS

FFmpeg és el programa que fa la feina pesada de codificació i segmentació. Disposa d'un *muxer* d'HLS (`-f hls`) que, a partir d'un vídeo d'entrada, genera les *playlists* i els segments [7]. El *backend* de MeTube construeix la comanda de manera dinàmica; el Llistat 3 mostra el cas complet d'una font que admet 240p, 480p, 720p i 1080p.

```
ffmpeg -i INPUT_VIDEO \
-filter_complex "[0:v]split=4[v0][v1][v2][v3];\
[v0]scale=w=trunc(oh*a/2)*2:h=240[v0out];\
[v1]scale=w=trunc(oh*a/2)*2:h=480[v1out];\
[v2]scale=w=trunc(oh*a/2)*2:h=720[v2out];\
[v3]scale=w=trunc(oh*a/2)*2:h=1080[v3out]" \
-map "[v0out]" -map "0:a:0" \
-map "[v1out]" -map "0:a:0" \
-map "[v2out]" -map "0:a:0" \
-map "[v3out]" -map "0:a:0" \
-c:v:0 libx264 -b:v:0 400k  -maxrate:v:0 450k  -bufsize:v:0 800k \
-c:v:1 libx264 -b:v:1 1000k -maxrate:v:1 1100k -bufsize:v:1 2000k \
```

```
-c:v:2 libx264 -b:v:2 2800k -maxrate:v:2 3000k -bufsize:v:2 5600k \
-c:v:3 libx264 -b:v:3 5000k -maxrate:v:3 5500k -bufsize:v:3 10000k \
-preset medium -c:a aac -b:a 128k \
-f hls -hls_time 6 -hls_playlist_type vod \
-hls_segment_filename "VIDEO_HLS_FOLDER/v%v_%03d.ts" \
-master_pl_name master.m3u8 \
-var_stream_map "v:0,a:0,name:240p v:1,a:1,name:480p v:2,a:2,name:720p v:3,a:3,name:1080p" \
"VIDEO_HLS_FOLDER/v%v.m3u8"
```

Llistat 3: Comanda FFmpeg (cas amb les quatre qualitats). `INPUT_VIDEO` i `VIDEO_HLS_FOLDER` són valors que el *backend* omple en cada cas.

Què fa cada bloc de la comanda:

- `split` **i** `scale` (dins de `-filter_complex`): divideixen el vídeo d'entrada en tantes còpies com qualitats i reescalen cada una a l'alçada indicada. L'expressió `w=trunc(oh*a/2)*2` calcula l'amplada mantenint la proporció de la imatge i la fa parell, perquè `libx264` (amb el format de píxel habitual) necessita dimensions parelles.
- `-map`: associa cada vídeo reescalat amb la pista d'àudio original (`0:a:0`), de manera que cada variant té vídeo i àudio.
- `libx264` **i AAC**: codifiquen el vídeo en H.264 i l'àudio en AAC (128 kbps), formats adequats per a la reproducció web.
- `-b:v`**,** `-maxrate` **i** `-bufsize`: controlen el *bitrate* de cada qualitat: el *bitrate* objectiu, el màxim permès i la mida del *buffer* sobre el qual s'aplica aquest màxim.
- `-preset medium`: compromís per defecte entre velocitat de codificació i eficiència de compressió.
- `-f hls` **i** `-hls_time 6`: generen HLS amb segments d'uns 6 segons. Amb `-hls_playlist_type vod` la *playlist* queda tancada amb `#EXT-X-ENDLIST`.
- `-master_pl_name` **i** `-var_stream_map`: creen la *master playlist* i relacionen cada parell vídeo-àudio amb el seu nom de qualitat i la seva *media playlist* (`v%v.m3u8`).

La Taula 1 resumeix els paràmetres de cada qualitat definits al *backend*. Abans de construir la comanda, el *backend* comprova la resolució del vídeo original amb `ffprobe` i descarta les qualitats que el vídeo no pot suportar (per exemple, no es genera 1080p si l'original és de 720p). Si el vídeo és més petit que 240p, es rebutja.

Qualitat	`-b:v`	`-maxrate`	`-bufsize`
240p	400k	450k	800k
480p	1000k	1100k	2000k
720p	2800k	3000k	5600k
1080p	5000k	5500k	10000k

Taula 1: Paràmetres de codificació de cada qualitat (extrets de `backend/src/app.js`).

2.6 *Adaptive Bit Rate* (ABR) i EWMA

L'ABR consisteix a escollir la qualitat del vídeo segons la connexió: si és dolenta, es demana una qualitat baixa; si és bona, una d'alta. En HLS això es fa **canviant de *playlist* en el segment següent**.

Una de les coses que s'han après durant el treball és que **l'adaptació es fa al costat del client**. El servidor no processa res de l'adaptació del *bitrate*: només proporciona la informació sobre el contingut disponible (la *master playlist* i les *media playlists*) i els segments. La decisió de quina variant es demana la pren el reproductor, que en el nostre cas és la biblioteca **hls.js** [8].

2.6.1 Per què una mitjana mòbil exponencial

Per estimar la velocitat de la connexió, hls.js utilitza una **mitjana mòbil ponderada exponencialment** (*Exponentially Weighted Moving Average*, EWMA). La motivació és que Internet és fonamentalment irregular, amb pics de connexió i *jitter* aleatori. Una mitjana normal seria massa sensible als valors nous i ignoraria de cop els valors anteriors; l'EWMA, en canvi, es pot ajustar a la situació, ja que permet modificar la sensibilitat als canvis nous. La fórmula és:

$$S_t = \alpha\, Y_t + (1 - \alpha)\, S_{t-1} \tag{1}$$

on S_t és l'estimació atenuada actual, S_{t-1} l'estimació atenuada anterior, Y_t el valor nou mesurat (*raw*) i α el factor d'atenuació. Si α és proper a 1, el valor nou té molt pes i l'estimació reacciona molt als canvis (s'assembla a una mitjana normal, i varia molt segons el punt actual). Si α és proper a 0, el valor nou pesa poc, es té molt en compte l'historial, i l'estimació és molt més estable però menys sensible.

2.6.2 EWMA ràpida i EWMA lenta

Segons el que s'ha estudiat, hls.js utilitza dues EWMA simultàniament:

- **EWMA ràpida** (*fast*): utilitza una α més gran per reaccionar quan la connexió cau amb rapidesa. S'utilitza per baixar la qualitat i evitar problemes de *throttling*.
- **EWMA lenta** (*slow*): utilitza una α més petita, per ser estable i adaptar-se a poc a poc a una connexió que millora. S'utilitza per pujar la qualitat.

Així, el reproductor baixa ràpid quan cal, però puja la qualitat amb prudència.

Nota important per a la interpretació dels resultats. Les corbes de l'EWMA ràpida i lenta que apareixen a les gràfiques de la Secció 5.3 **no** són els valors interns de hls.js. Són corbes complementàries calculades pel nostre codi a partir de la mostra de velocitat de cada segment, amb semivides de 3 s (ràpida) i 9 s (lenta), de manera que l'α de cada mostra és $\alpha = 1 - e^{-\ln 2 \cdot d / T_{1/2}}$, on d és el temps de descàrrega del segment i $T_{1/2}$ la semivida. Cal distingir-les de l'estimació agregada que exposa hls.js, que també es representa.

3 Escenari 1: descripció

3.1 Part comuna: infraestructura de xarxa de l'organització

La part comuna del Projecte 1 consisteix a construir la infraestructura d'una organització unida a la resta mitjançant un ISP, de manera que entre tots es forma una "petita Internet" dins del laboratori. Cada organització ha de tenir com a mínim: un *router* que la uneix amb l'exterior, un prefix de xarxa IP, dues xarxes IP, una xarxa Ethernet, un servidor DHCP, un servidor DNS local, un servidor web accessible des de l'exterior amb un nom DNS públic, i clients.

L'escenari complet incloïa més màquines, però la majoria ja no existeixen. Només han quedat tres elements (veieu la Fig. 3):

- L'estació **1.8.8.2**, a la primera xarxa (**1.8.8.0/25**), que fa de client.
- El servidor **1.8.8.130**, a la segona xarxa (**1.8.8.128/25**). És el servidor de MeTube i també el de BIND9.
- El *router* OpenWrt, que uneix les dues xarxes i està connectat a la xarxa exterior 20.20.20.0/24. La seva interfície a la primera xarxa té l'adreça 1.8.8.1, la de la segona xarxa té la 1.8.8.129 i la de la xarxa exterior la 20.20.20.8.

Totes dues màquines tenen la configuració de xarxa estàtica. El nom públic del servidor és **www.ponent.cat**, que apunta a 1.8.8.130 i es gestiona al servidor autoritatiu d'en Pau.

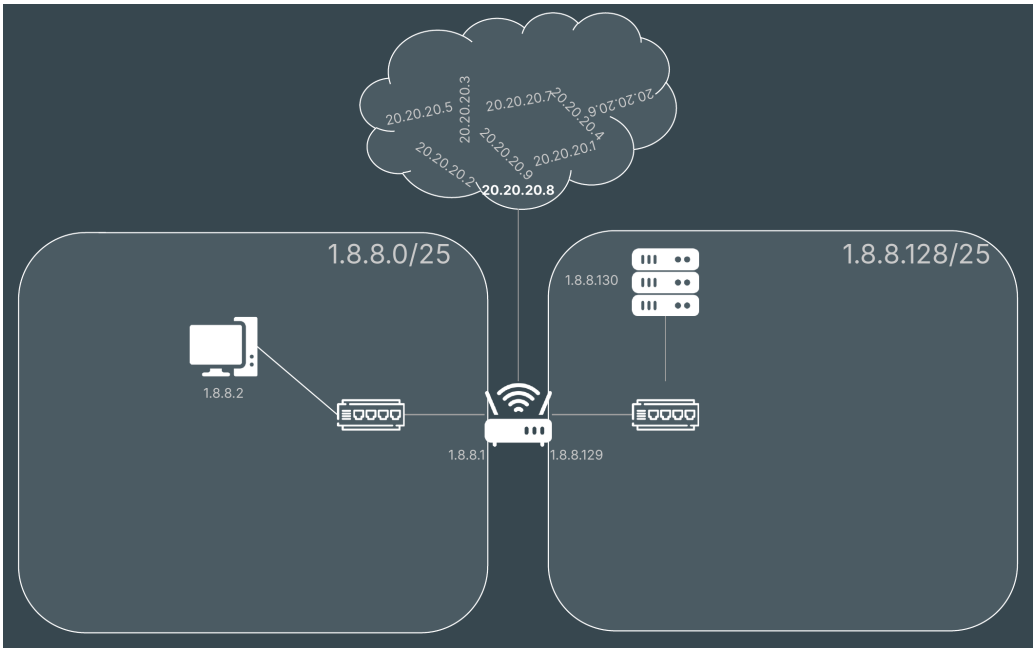


Figura 3: Escenari de xarxa actual: client (1.8.8.2), servidor (1.8.8.130) i *router* OpenWrt connectat a l'exterior.

3.2 Part específica: plataforma *MeTube*

La part específica és una plataforma de *streaming* anomenada **MeTube**, una petita còpia de YouTube que utilitza HLS. Està formada per un servidor (*backend*) i una aplicació web (*frontend*); la Fig. 4 en mostra l'arquitectura.

- ***Backend***: Express.js [9] (Node.js) amb SQLite per guardar les metadades dels vídeos (identificador, títol, descripció, usuari, durada, data de publicació, estat i adreça HLS). Utilitza FFmpeg i `ffprobe` per codificar i segmentar els vídeos.
- ***Frontend***: aplicació en React que mostra la llista de vídeos, permet pujar-ne i reprodueix el vídeo escollit amb la biblioteca hls.js [8].
- **Divisió de la feina**: la part del servidor fa la codificació i la segmentació, i serveix els fitxers estàtics (*master*, *playlists* i segments). La part del client llegeix les *playlists*, demana els segments i executa l'ABR.

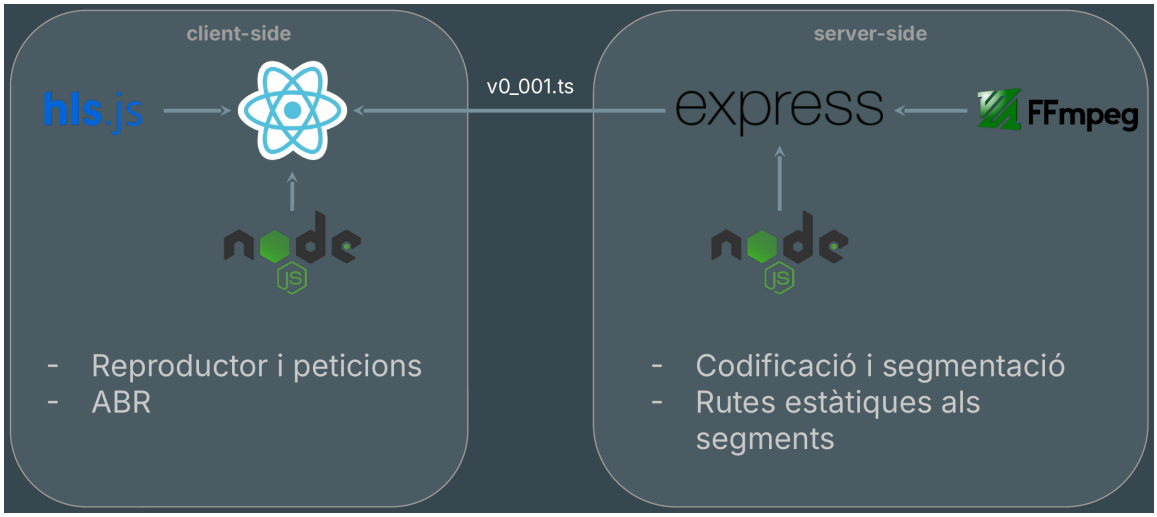


Figura 4: Arquitectura de MeTube: la part del client (reproductor i ABR) i la del servidor (codificació, segmentació i rutes estàtiques als segments).

Cal tenir en compte que el servidor de Node del *frontend* és un servidor de desenvolupament, que simplement s'encarrega de "compilar" React de manera dinàmica. L'he fet servir per a les proves, però en un entorn de producció caldria compilar l'aplicació i servir-la de forma estàtica.

Dos fluxos. Un usuari pot *veure* o *pujar* un vídeo. Quan en puja un, el client envia el vídeo original en MP4 al servidor; el servidor respon de seguida amb estat *transcoding* i fa la feina pesada en segon pla (canvi de resolucions, codificació i segmentació). Quan acaba, l'estat passa a *ready* (o *failed* si hi ha un error) i el vídeo original s'esborra del servidor, de manera que només es conserven els fitxers HLS. Quan un client veu un vídeo, descarrega la *master playlist*, les *playlists* i els segments des del servidor. Tot el servei funciona sobre HTTP: no s'han utilitzat TLS ni certificats.

Eines d'experimentació. Per estudiar l'ABR s'ha afegit al *frontend* una ruta independent (`/experiment/<id>`) que no modifica la reproducció normal. Crea el seu propi reproductor hls.js i enregistra una fila per cada segment descarregat en un fitxer CSV, que es descarrega automàticament en acabar la reproducció. Les columnes principals són: temps transcorregut, qualitat (nivell, resolució i *bitrate* anunciat), durada del segment, temps i *bytes* descarregats, velocitat de descàrrega (`download_bps`), l'estimació agregada de hls.js i les dues EWMA complementàries. A més, hi ha un *script* en Python que genera les gràfiques a partir del CSV, i un *script* de *shell* que limita l'ample de banda amb `tc` [10] seguint un perfil de fases. Els detalls es donen a les seccions 4 i 5.

4 Escenari 1: construcció

4.1 Part comuna: màquines virtuals, *router* i DNS

4.1.1 Màquines i *router*

Les màquines virtuals es van crear a partir de la ISO oficial d'Arch Linux, utilitzant l'eina `archinstall` per instal·lar ràpidament un sistema senzill. El servidor no té entorn gràfic i el client sí. En totes dues he utilitzat `neovim` per editar els fitxers.

Al servidor he instal·lat els paquets `bind` i `nodejs` (amb `npm`) per a l'allotjament web, `ffmpeg`

per a la codificació i la segmentació, i `python` per a l'*script* que utilitza `tc`. Al client he instal·lat `chromium` per utilitzar la web. Les dues màquines tenen la configuració de xarxa estàtica, amb les opcions necessàries.

No hi ha tallafoc: el d'OpenWrt ve activat per defecte i el vaig haver de desactivar. Me'n vaig adonar fent proves amb en Arnau el dia de l'últim lliurament de seguiment.

4.1.2 Problema de connectivitat entre màquines virtuals

Les dues màquines tenien la interfície `enp0s3`, connectada a una xarxa interna (*internal network*), i una interfície `enp0s8` per instal·lar paquets. Com que les adreces IP que fem servir són adreces públiques genèriques (rang d'empresa 1.8.8.0/24, 20.20.20.0/24...), no s'obtenia resposta, i per alguna raó es feia servir la interfície equivocada, tot i especificar-ne una altra. En desactivar la interfície `enp0s8`, la connexió va funcionar correctament.

4.1.3 Servidor DNS recursiu (BIND9)

La configuració del DNS recursiu es va fer seguint les instruccions d'en Pau. L'única diferència és que, en utilitzar una distribució diferent, els fitxers de BIND no eren a `/etc/bind` sinó que la configuració és a `/etc/named.conf`. El contingut dels fitxers és pràcticament idèntic.

4.2 Part específica: construcció de MeTube

4.2.1 Cronologia del treball

Es descriu a continuació com va avançar el treball, de manera cronològica. Cal tenir en compte que la comprensió d'HLS no va ser immediata: primer es va construir la plataforma i després es va entendre com funcionava per dins.

1. ***Backend* i *frontend* des del principi.** Es va començar directament amb una aplicació web amb Express per al *backend*, React per al *frontend* i HLS per a la part de *streaming*. La part de FFmpeg (la comanda de codificació i segmentació) es va fer amb ajuda.
2. **Màquines i xarxa.** Es van crear i configurar les màquines. En aquesta fase es va detectar que no es connectaven entre elles (vegeu la secció anterior).
3. **Pàgina funcional.** Es va aconseguir una pàgina web completament funcional amb HLS.
4. **Descobriment de l'ABR.** Es va descobrir que l'adaptació es fa al costat del client i que el servidor només proporciona la informació del contingut disponible. Això té sentit, però no s'havia plantejat fins aleshores. Després es va estudiar l'EWMA utilitzada per hls.js (Secció 2.6).
5. **Planificació de l'experiment.** Es va veure interessant preparar un vídeo molt llarg (1–1,5 h) i un *script* que generés gràfiques dels valors de les EWMA, de la velocitat, etc.
6. **Eines d'anàlisi i experiment.** Es va crear la ruta d'experiment del *frontend*, la sortida en CSV, l'*script* de gràfiques i el perfil de limitació de xarxa amb `tc`.
7. **Experiment complet i presentació.** Es va fer l'enregistrament d'un vídeo d'una hora i vuit minuts i es va presentar el resultat a classe, juntament amb una demostració de l'aplicació.

Problemes amb el limitador de xarxa. En les proves inicials amb el limitador de xarxa vaig tenir problemes al client, perquè `tc` actua sobre el trànsit de sortida. Per això el vaig posar al servidor. Durant uns 15 minuts vaig pensar que no funcionava, fins que em vaig adonar que Chromium informava de respostes `304 Not Modified`: els segments "descarregats" venien de la memòria cau. En esborrar-la, el limitador va funcionar correctament. Després vaig deixar que s'executés l'*script* i vaig generar les gràfiques.

4.2.2 Instal·lació i execució

El servidor té tres serveis: el *backend* (Node.js i Express), el *frontend* (un servidor de desenvolupament de Node) i BIND9. El *backend* requereix Node.js i FFmpeg (amb `ffmpeg` i `ffprobe`). S'instal·len les dependències i s'executa amb:

```
npm install
npm start      # escolta al port 3000
```

Llistat 4: Posada en marxa del *backend* (a `backend/`).

El *backend* escolta al port 3000 i el CORS accepta peticions de `http://www.ponent.cat`. El *frontend* s'executa amb el servidor de desenvolupament de Node, tal com s'ha explicat a la secció anterior. No hi ha cap certificat: tot va sobre HTTP.

Per pujar un vídeo, l'API rep una petició `POST /api/upload` (*multipart*) amb el fitxer, el títol, la descripció i el nom d'usuari. El servidor respon amb el codi 202 mentre es transcodifica, i l'estat es pot consultar a `GET /api/videos/:id`. La *master playlist* queda disponible a `/hls/<id>/master.m3u8`.

4.2.3 Limitació de la xarxa amb `tc`

Per veure l'ABR amb claredat, es limita l'ample de banda per fases amb `tc` [10]. L'*script* funciona sobre el servidor i la seva interfície de xarxa de sortida. Llegeix un perfil en CSV (`70min.csv`) amb les columnes `duration_seconds,rate_mbit,label`; per a cada fase aplica una disciplina de cua `tbf` (*token bucket filter*) amb la taxa corresponent i espera la durada de la fase. El perfil té 56 fases i dura 70 minuts (vegeu l'Annex A). El Llistat 5 mostra la part essencial.

```
tc qdisc replace dev "$IFACE" root tbf rate "${rate}mbit" burst 512kb latency 150ms
sleep "$duration"
```

Llistat 5: Part essencial de l'*script* de limitació: per a cada fase del perfil.

4.2.4 Generació de les gràfiques

Un *script* de Python (`plot_abr_profiles.py`) llegeix el CSV de resultats i el del perfil de xarxa, i genera una gràfica per a cada troç de 10 minuts. Cada gràfica mostra la velocitat mesurada, les dues EWMA, l'estimació de hls.js, la qualitat seleccionada i, sobre el fons, les fases del perfil amb la seva taxa.

4.2.5 Execució de l'experiment

La metodologia seguida és la següent. Es prepara un vídeo llarg (uns 68 minuts) per tenir un conjunt experimental prou gran. S'obre la ruta `/experiment/<id>` i es prem *Start / restart*

recording; en paral·lel s'executa l'*script* de `tc`. Quan acaba el vídeo, es descarrega el CSV i es generen les gràfiques, i després s'amplia un tros curt per analitzar-lo bé.

L'experiment no es va repetir. El vídeo és un vídeo genèric descarregat de YouTube en la qualitat màxima i pujat a través de la mateixa web.

5 Escenari 1: anàlisi

5.1 Part comuna: proves de funcionament

El dia de les proves de funcionament de la part comuna no vaig portar les màquines, de manera que no es van poder provar. Tot i això, unes setmanes abans es van fer algunes proves amb `ping` amb en Arnau, i l'allotjament funcionava correctament.

5.2 Part específica: comprovació del funcionament d'HLS

Es va comprovar que la plataforma funciona correctament: es poden pujar vídeos, que passen d'estat *transcoding* a *ready*, i reproduir-los des dels clients. A la demostració feta a classe es va observar el flux real de reproducció HLS amb les eines de desenvolupador del navegador, a la pestanya de xarxa (Fig. 5), amb una limitació de velocitat simulada ("Slow 4G") per veure el canvi de qualitat. La web va sobre HTTP.

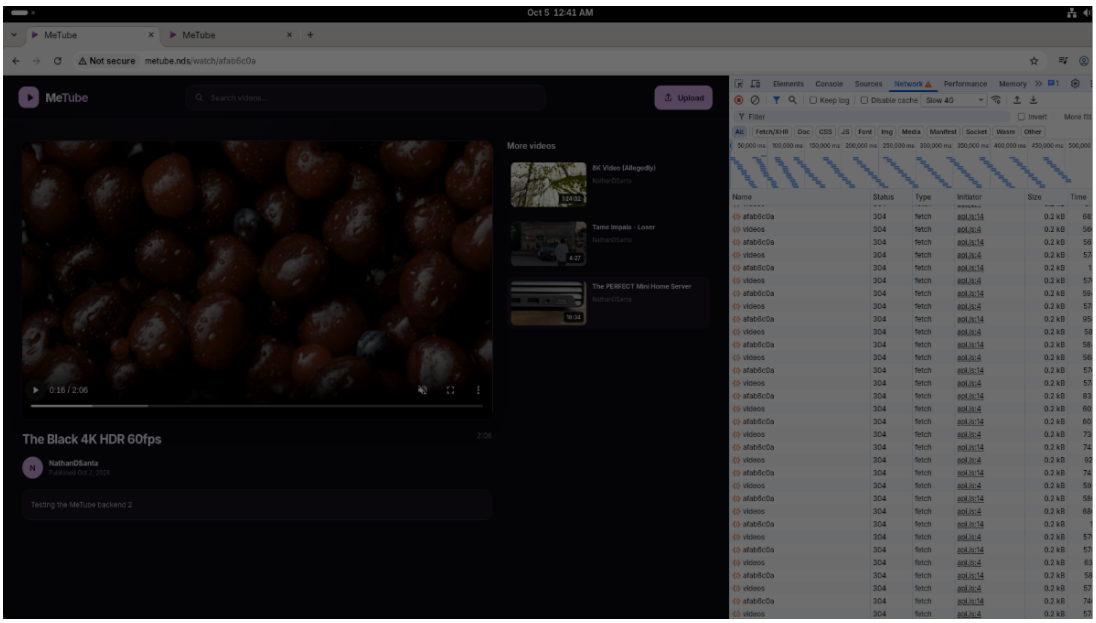


Figura 5: Reproducció d'un vídeo a MeTube amb la pestanya de xarxa del navegador oberta.

No s'ha fet cap anàlisi amb Wireshark. Les peticions es van observar amb les eines de desenvolupador de Chromium.

5.3 Part específica: experiment de l'ABR

5.3.1 Disseny de l'experiment

L'experiment consisteix a reproduir un vídeo llarg mentre es limita l'ample de banda de la xarxa seguint un perfil de 56 fases (Annex A). El perfil comença amb 12 Mbps durant mig minut, baixa fins a 0,05 Mbps als punts més durs (per exemple, 0,08 Mbps entre els minuts 29,8 i 30,0, i 0,05 Mbps entre el 46,2 i el 46,3) i torna a pujar fins a 15 Mbps en una fase curta, de manera que es combinen fases estables amb caigudes i recuperacions brusques. Això permet observar quan el reproductor puja i baixa de qualitat.

5.3.2 Dades registrades

El CSV de l'execució conté 525 mostres de segment i cobreix 4 088 segons (uns 68 minuts) de reproducció. Les quatre qualitats de la *master playlist* es van fer servir en algun moment: 240p, 480p, 720p i 1080p. Entre segments consecutius, el nivell de qualitat va canviar 50 vegades. Els *bitrates* anunciats a la *master playlist* d'aquest vídeo són els de la Taula 2.

Nivell	Resolució	*Bitrate* anunciat (bps)
0	240p	676 980
1	480p	1 414 602
2	720p	3 587 039
3	1080p	6 337 916

Taula 2: *Bitrates* anunciats de cada nivell, segons el CSV de l'experiment.

La comanda de FFmpeg demana segments de 6 segons, però durant l'experiment es va veure que els segments duraven de mitjana uns 8,3 segons. No ho vaig notar mentre feia les proves i no estic segur del que va provocar aquesta diferència.

5.3.3 Resultats

Les figures 6 i 7 mostren, per a dos troços de 10 minuts, la velocitat de descàrrega mesurada, les corbes EWMA ràpida i lenta, l'estimació de hls.js, la resolució seleccionada i el *bitrate* de la qualitat. A l'eix horitzontal hi ha el temps transcorregut, i les franges de color del fons són les fases del perfil de xarxa, amb la seva taxa. Es pot veure clarament que el valor d'ample de banda llegit és molt erràtic.



Figura 6: Resultat de l'ABR entre els minuts 10 i 20 de l'experiment. Les mostres de velocitat més altes es retallen al percentil 90 per poder veure bé la resta de corbes.



Figura 7: Resultat de l'ABR entre els minuts 40 i 50 de l'experiment. Les mostres de velocitat més altes es retallen al percentil 90.

6 Resum, conclusions i treball futur

6.1 Què s'ha fet

Part comuna. S'ha construït l'escenari de xarxa de l'organització amb màquines virtuals Arch Linux i un *router* OpenWrt, i s'ha configurat un servidor DNS recursiu amb BIND9 que depèn d'un servidor autoritatiu fet per en Pau.

Part específica. S'ha creat MeTube, una plataforma de *streaming* amb HLS (Express, SQLite, React, hls.js i FFmpeg); s'han creat unes eines per enregistrar el comportament de l'ABR i per generar gràfiques; i s'ha fet un experiment d'una hora i vuit minuts amb un perfil de xarxa de 56 fases.

6.2 Problemes trobats

- Les màquines virtuals no es connectaven entre elles: el dispositiu feia servir la interfície equivocada. Es va solucionar desactivant la interfície `enp0s8`.
- El tallafoc d'OpenWrt, activat per defecte, es va haver de desactivar.
- La configuració de BIND9 era en un altre lloc a causa de la diferència entre distribucions (Ubuntu i Arch Linux).
- Vaig pensar que `tc` no funcionava correctament, però en realitat Chromium agafava els trossos de la memòria cau.
- No em vaig adonar que els trossos no duraven 6 segons.
- L'adaptació m'ha semblat lenta en les baixades de qualitat. És possible que la configuració de la comanda de FFmpeg no fos la millor.

6.3 Conclusions i lliçons apreses

- HLS funciona sobre HTTP, de manera que reutilitza infraestructura existent (servidors, CDN) i no necessita ports addicionals, a diferència dels protocols anteriors.
- Els problemes clàssics del *streaming* són que els paquets no arriben amb temps constant i que la connexió pot provocar interrupcions. Els protocols anteriors necessitaven infraestructura especialitzada i permisos o ports oberts addicionals.
- L'ABR fa que les connexions dolentes demanin qualitats més baixes (i que torni a pujar si la connexió millora). El processament es fa al client; el servidor només serveix el contingut.
- La decisió d'ABR és al reproductor (hls.js), que fa servir EWMA per suavitzar les mesures irregulars de la xarxa.

6.4 Treball futur

- Veure altres opcions més actuals, com ara HLS amb MPEG i MP4, perquè funcionen de manera diferent de l'HLS normal.
- Explorar altres mètodes més avançats que no conec.

- Repetir l'experiment més d'una vegada per comparar resultats.
- Ampliar el treball a la videoconferència amb Jitsi Meet, que no s'ha pogut fer per falta de temps.

7 Treball en grup, dedicació i suggeriments

Aquest ha estat un treball individual. Durant les hores de classe (18,5 h) només vaig fer el *game plan* del projecte; la major part de la feina es va fer a casa. Fora de clase foren aproximadament 20h

Gràcies a la IAG no he hagut de dedicar tantes hores a feina menys important per a l'anàlisi del protocol, com ara:

- L'*script* per controlar la xarxa. Els detalls de com funciona no són importants, ja que el que es vol veure és com s'adapta HLS; ho hauria pogut fer manualment amb les eines de desenvolupador de Chromium, com vaig fer a classe, però volia un rang de dades més ampli per a l'anàlisi.
- El *frontend* de l'aplicació. Es veu relativament bé, però hauria pogut fer exactament el mateix amb només un `<h1>` amb el nom del vídeo i el reproductor hls.js; així, però, és més visual i agradable.

8 Ús d'eines d'Intel·ligència Artificial Generativa (IAG)

Pel que fa a l'ús d'IAG, només s'ha utilitzat per generar codi *boilerplate*, com ara el codi del *frontend*, que només té com a objectiu l'aparença (excloent les poques línies que s'utilitzen per fer el visualitzador hls.js); per generar l'*script* de control de la xarxa; i per crear amb Python les gràfiques. Tenia les dades en CSV, de manera que amb temps hauria pogut fer el mateix amb un full de càlcul, però així he pogut dedicar menys temps a l'anàlisi en si.

També s'ha fet servir per cercar informació quan estava amagada entre pàgines de documentació, però tot s'ha verificat amb altres fonts.

Referències

[1] Oracle, *Oracle VM VirtualBox* (lloc web oficial d'Oracle VirtualBox) [en línia]. Disponible a: `https://www.virtualbox.org/`.

[2] Arch Linux, *Arch Linux* (lloc web oficial) [en línia]. Disponible a: `https://archlinux.org/`.

[3] OpenWrt Project, *OpenWrt* (lloc web oficial) [en línia]. Disponible a: `https://openwrt.org/`.

[4] Internet Systems Consortium, *BIND 9 Administrator Reference Manual* [en línia]. Disponible a: `https://bind9.readthedocs.io/`.

[5] J.F. Kurose, K.W. Ross, *Computer networking. A top-down approach featuring the Internet*, Addison-Wesley, 8a ed., 2022.

[6] R. Pantos, W. May, *HTTP Live Streaming*, RFC 8216, IETF, agost de 2017 [en línia]. Disponible a: `https://www.rfc-editor.org/rfc/rfc8216`.

[7] FFmpeg, *FFmpeg Formats Documentation*, secció *hls* [en línia]. Disponible a: `https://ffmpeg.org/ffmpeg-formats.html`.

[8] video-dev, *hls.js* (repositori i documentació de l'API) [en línia]. Disponible a: `https://github.com/video-dev/hls.js`.

[9] OpenJS Foundation, *Express - Node.js web application framework* [en línia]. Disponible a: `https://expressjs.com/`.

[10] *tc(8) – show / manipulate traffic control settings*, Linux manual page [en línia]. Disponible a: `https://man7.org/linux/man-pages/man8/tc.8.html`.

A Annex A: perfil de limitació de xarxa utilitzat

La Taula 3 recull les 56 fases del perfil de limitació de l'experiment (taxa i interval en minuts), tal com apareix al fitxer `70min.csv`.

Taula 3: Perfil de limitació d'ample de banda (56 fases).

#	Taxa (Mbps)	Inici (min)	Final (min)	Durada (min)
1	12	0,0	0,5	0,5
2	8	0,5	1,2	0,8
3	4	1,2	2,2	1,0
4	1,5	2,2	2,7	0,4
5	0,5	2,7	2,9	0,2
6	3,5	2,9	3,7	0,8
7	6	3,7	5,7	2,0
8	10	5,7	6,7	1,0
9	5	6,7	9,2	2,5
10	2	9,2	10,0	0,8
11	1,2	10,0	10,7	0,7
12	0,4	10,7	11,0	0,3
13	0,1	11,0	11,2	0,2
14	1,8	11,2	11,7	0,5
15	4,5	11,7	13,2	1,5
16	7,5	13,2	15,7	2,5
17	2,8	15,7	17,0	1,3
18	0,8	17,0	17,7	0,7
19	0,3	17,7	18,0	0,3
20	3	18,0	19,2	1,2
21	6,5	19,2	22,2	3,0
22	8,5	22,2	25,0	2,8
23	1	25,0	25,5	0,5
24	15	25,5	25,8	0,2
25	0,6	25,8	26,1	0,3
26	2,2	26,1	26,8	0,8
27	4	26,8	27,8	1,0
28	9	27,8	29,3	1,5
29	0,2	29,3	29,8	0,4
30	0,08	29,8	30,0	0,2
31	1,5	30,0	30,6	0,6
32	5,5	30,6	32,6	2,0
33	10	32,6	35,6	3,0
34	3,2	35,6	37,4	1,8
35	0,7	37,4	38,2	0,8

#	Taxa (Mbps)	Inici (min)	Final (min)	Durada (min)
36	6	38,2	40,8	2,7
37	8	40,8	45,0	4,2
38	1,8	45,0	45,7	0,7
39	0,5	45,7	46,2	0,5
40	0,05	46,2	46,3	0,2
41	0,3	46,3	46,7	0,3
42	2,5	46,7	47,7	1,0
43	5	47,7	50,2	2,5
44	7	50,2	53,5	3,3
45	2	53,5	55,0	1,5
46	0,4	55,0	55,5	0,5
47	0,1	55,5	55,7	0,2
48	3	55,7	56,7	1,0
49	8,5	56,7	60,0	3,3
50	11	60,0	61,5	1,5
51	4,5	61,5	62,5	1,0
52	1,2	62,5	63,2	0,7
53	0,3	63,2	63,5	0,3
54	2,5	63,5	64,3	0,8
55	6	64,3	67,3	3,0
56	10	67,3	70,0	2,7